

Construye | Empaquetar modelo del proyecto MiBici como API REST en contenedor

Reporte de evidencias técnicas · MLflow Model Registry

Iván Plata Garnica

Axel Fernando Rábago Díaz

Mónica María Ramírez Mejía

TC5061 Operaciones de aprendizaje automático · Grupo 10 · Equipo 46

08 de noviembre de 2026

Repositorio: https://github.com/iplatamx/TC5061-Proyecto-Equipo46

Pull Request #12 Axel Fernando: https://github.com/iplatamx/TC5061-Proyecto-Equipo46/pull/12

Resumen ejecutivo

En la Semana 8 iniciamos el despliegue del proyecto MiBici. Mi contribución se concentró en recuperar el modelo P1 desde MLflow Model Registry, definir las entradas de inferencia, comprobar que el modelo realmente predice y dejar el componente listo para su integración posterior con FastAPI y Docker o Podman. El documento sigue la estructura del reporte de la Semana 7: explicación del trabajo, resultados verificables y anexos con capturas.

El desarrollo quedó publicado en la rama feature/semana8-mlflow-axel y se creó el Pull Request #12 hacia main. El modelo se encuentra en el MLflow local utilizado para las pruebas; no se incluyen sus artefactos binarios en GitHub.

1. Alcance y relación con la rúbrica

Criterio de evaluación	Puntos	Estado verificable
Recuperar modelo del Model Registry	20	Implementado; registro y predicción comprobados
API REST en FastAPI + validación	30	Contrato preparado; endpoint a cargo de otro integrante
Contenerización Docker/Podman	30	Pendiente de integración del equipo
README y ejemplo de consumo	10	Documento técnico parcial; README final pendiente
Colaboración mediante Git	10	Commit y PR #12 publicados; revisión/merge pendientes

2. Ficha del modelo recuperado

Elemento	Dato comprobado
Modelo	mibici-equipo46-p1-demanda
Candidato	P1-HGB-02 · HistGradientBoostingRegressor
Versión y alias	Versión 1 · champion
MAE de validación	0.735193
Versión de datos	129a4a57efc7
Salida de prueba	3.923333879487776
Seguimiento	MLflow 3.6.0 · servidor local en 127.0.0.1:5000

3. Archivos implementados

Ruta	Responsabilidad
mibici/serving/contract.py	Ocho características de entrada y validaciones
mibici/serving/model_loader.py	Resolver alias, cargar versión y generar predicciones
mibici/serving/register_p1.py	Registrar el mejor candidato P1 con alias champion
mibici/serving/verify_model.py	Comprobar inferencia real desde el registro
tests/test_serving_*.py	Pruebas unitarias y prueba de integración
docs/Semana8_Axel_ModelRegistry.md	Guía técnica para integrar y verificar el módulo

4. Procedimiento desarrollado

4.1. Preparación y recuperación de los datos

La ejecución inicial de la ingesta falló al solicitar nomenclatura_2026_09.csv (HTTP 404). Para continuar, actualicé el catálogo a nomenclatura_2026_10.csv. La verificación posterior confirmó 20 archivos consistentes, entre ellos nueve archivos de viajes procesados. Esta actualización produjo la versión de datos 129a4a57efc7, distinta de d038fe16493d, utilizada en la entrega anterior.

4.2. Corrección del entorno de entrenamiento

Durante la reproducción del entrenamiento base encontré primero la ausencia del paquete deepchecks y después un error por el uso de np.Inf, eliminado de NumPy 2.x. El entorno comprobado utilizó Python 3.11.9, NumPy 1.26.4 y Deepchecks 0.19.1. La comprobación de dependencias concluyó sin conflictos declarados (pip check).

4.3. Experimentos, registro e inferencia

Se logró ejecutar el entrenamiento base P1 con MLflow Projects. Posteriormente se entrenaron los candidatos avanzados y se seleccionó P1-HGB-02 por su MAE de validación. Registré el candidato como versión 1 de mibici-equipo46-p1-demanda, asociado al alias champion. Una comprobación independiente cargó la versión resuelta mediante models:/ y generó una predicción numérica.

4.4. Contrato de entrada definido para FastAPI

Variable	Interpretación
station_id	Identificador de la estación
departures_this_hour	Salidas de la hora observada
arrivals_this_hour	Llegadas de la hora observada
departures_mean_last_24h	Media de salidas en 24 horas
departures_mean_last_168h	Media de salidas en 168 horas
target_hour	Hora objetivo
target_dayofweek	Día de la semana objetivo
target_is_weekend	Indicador de fin de semana

El módulo devuelve predicciones numéricas y valida el contrato; la validación HTTP con Pydantic y el endpoint POST /predict serán implementados por el compañero responsable de FastAPI.

5. Resultados de las pruebas

Verificación	Resultado	Interpretación
Contrato de datos	13 passed	Campos, tipos y valores inválidos
Cargador MLflow	2 passed	Alias, firma y predicción simulada
Integración Registry	1 passed	Carga e inferencia con modelo real
Pruebas generales	37 passed, 2 warnings	No se observaron fallos en el conjunto general
Dependencias	No broken requirements found	Sin conflictos declarados por pip

Las 16 pruebas específicas se ejecutaron juntas y terminaron correctamente. Las 37 pruebas de la ejecución general no deben sumarse automáticamente a las 16, porque hay solapamiento entre ambos conjuntos. Las dos advertencias observadas corresponden a avisos de obsolescencia de Typer/Click y no son errores de prueba.

6. Observación del profesor: reproducibilidad

La observación previa indicaba que los resultados del proyecto no habían podido replicarse. En esta revisión identifiqué y corregí en mi entorno tres obstáculos concretos: un archivo externo no disponible, la falta de Deepchecks y su incompatibilidad con NumPy 2.x. Después de los ajustes, el entrenamiento base llegó a MLflow con estado succeeded y MAE de prueba 1.4756.

Este resultado confirma que el procedimiento pudo ejecutarse en mi entorno; no demuestra una reproducción numéricamente idéntica de la entrega anterior, ya que la versión de datos cambió. Por transparencia, conservé una copia del manifiesto 129a4a57efc7 en docs/evidencias/semana8/, sin sustituir de manera silenciosa el manifiesto histórico del repositorio.

7. Trabajo colaborativo y pasos siguientes

El commit 8e2a57a reúne 13 archivos y fue publicado en feature/semana8-mlflow-axel; la solicitud de integración quedó abierta como PR #12. El equipo deberá completar la API FastAPI, empaquetar el servicio con Docker o Podman y documentar la ejecución HTTP desde el contenedor. Para que esto funcione en otro equipo será necesario proporcionar acceso al Model Registry y sus artefactos, o empaquetar un modelo exportado de versión controlada.

8. API REST con FastAPI y validación de entradas

Esta sección documenta los puntos 2 y 3 de la actividad: la construcción de la API REST que expone el modelo P1, la definición de los esquemas de entrada y salida con Pydantic y la comprobación de que el servicio responde con el modelo real recuperado desde MLflow Model Registry. La API utiliza el componente de recuperación descrito en las secciones anteriores (MiBiciModelService), sin duplicar su lógica.

El desarrollo quedó publicado en la rama feature/semana8-api-fastapi y se creó el Pull Request #13 hacia main.

8.1. Endpoints implementados

Endpoint	Función
GET /health	Estado del servicio; indica si el modelo quedó cargado
GET /model-info	Nombre, versión y variables del modelo en uso
POST /predict	Predicción de salidas en la hora siguiente para una estación
POST /predict/batch	Predicción para varias estaciones en una sola solicitud
GET /docs	Documentación interactiva generada por FastAPI

La API carga el modelo una sola vez al iniciar y reutiliza la versión resuelta en todas las solicitudes; no consulta MLflow en cada petición. Si el registro no está disponible al iniciar, el servicio arranca de todos modos y /health responde 503, lo que permite identificar el problema dentro del contenedor.

8.2. Validación de entradas y salidas

Variable	Regla de validación
station_id	Entero mayor que 0
departures_this_hour	Entero mayor o igual que 0
arrivals_this_hour	Entero mayor o igual que 0
departures_mean_last_24h	Número mayor o igual que 0
departures_mean_last_168h	Número mayor o igual que 0
target_hour	Entero entre 0 y 23
target_dayofweek	Entero entre 0 (lunes) y 6 (domingo)
target_is_weekend	0 o 1, consistente con target_dayofweek

El esquema de entrada replica el contrato definido en la sección 4.4: contiene las mismas ocho variables, en el mismo orden, y rechaza campos adicionales, campos faltantes y valores de texto donde se espera un número. Una entrada inválida recibe HTTP 422 con el campo y el motivo del error, sin llegar al modelo.

El esquema de salida define station_id, predicted_departures_next_hour, model_name y model_version. La predicción nunca es negativa, igual que en la evaluación original de P1, e incluir la versión permite identificar qué modelo generó cada respuesta.

8.3. Archivos implementados

Ruta	Responsabilidad
api/main.py	Aplicación FastAPI, endpoints y manejo de errores
api/schemas.py	Esquemas Pydantic de entrada y salida
tests/test_api.py	Pruebas de la API con un servicio simulado
requirements.txt	Dependencias de la API con versión fija
docs/Semana8_API_FastAPI.md	Guía técnica para ejecutar y verificar la API

8.4. Procedimiento de verificación

Para probar la API con el modelo real se preparó un entorno nuevo y un servidor de MLflow sin registros previos. La instalación de dependencias falló primero con Python 3.14, porque NumPy 1.26.4 no está disponible para esa versión; con Python 3.11.9, la versión indicada en la sección de la Semana 7 del README, la instalación concluyó sin errores. PyTest se instaló por separado, como indica el README.

Después se reprodujo el procedimiento del repositorio: ingesta con MLflow Projects, búsqueda de los candidatos P1, registro con register_p1 y verificación con verify_model. Finalmente, la API se levantó con Uvicorn y se consultó desde la documentación interactiva de FastAPI.

8.5. Resultados de las pruebas

Verificación	Resultado	Interpretación
Pruebas de la API	15 passed, 1 warning	Aviso de obsolescencia de starlette/anyio; no es

		un error de prueba
Ingesta	129a4a57efc7	3,361,307 viajes, 9 de 9 meses, 470 estaciones
Búsqueda P1	P1-HGB-02 · MAE 0.735193	Mismo candidato seleccionado en la sección 2
Registro	Versión 1 · champion	Alias resuelto correctamente
Recuperación	3.923333879487776	Predicción de verify_model
GET /health	200	Modelo cargado desde el registro
POST /predict	200 · 3.9233	Coincide con verify_model, redondeado a cuatro decimales
POST /predict con target_hour = 24	422	Entrada rechazada antes de llegar al modelo

La versión de datos, el candidato, la versión registrada y la predicción coinciden con la ficha de la sección 2. Este resultado confirma que el procedimiento documentado pudo ejecutarse en una segunda computadora y produjo un modelo con los mismos resultados para la versión de datos 129a4a57efc7.

8.6. Ejemplo de consumo

Solicitud enviada a POST /predict y respuesta obtenida con el modelo real:

Campo	Valor
station_id	1
departures_this_hour	5
arrivals_this_hour	4
departures_mean_last_24h	3.5
departures_mean_last_168h	4.2
target_hour	9
target_dayofweek	1
target_is_weekend	0

Campo de respuesta	**Valor**
station_id	1
predicted_departures_next_hour	3.9233
model_name	mibici-equipo46-p1-demanda
model_version	1

Actualización de la actividad integradora

Estado actualizado: esta ampliación complementa el reporte previo. La tabla de alcance de la página 2 y la nota de trabajo pendiente de la página 3 corresponden a un corte anterior. Los puntos 4, 5 y 6 se documentan aquí; los puntos 1, 2 y 3 ya están descritos en las secciones anteriores.

9. Punto 4 - Containerfile compatible con OCI

El archivo **Containerfile** se agregó en la raíz del repositorio. La imagen usa Python 3.11 sobre Debian slim, instala las dependencias fijadas en `requirements.txt` y el runtime `libgomp1` requerido por scikit-learn. Copia el código de la API y el servicio del modelo, ejecuta con el usuario sin privilegios 10001 y publica el puerto 8000. Uvicorn inicia `api.main:app`.

```
FROM python:3.11-slim-bookworm

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    MLFLOW_TRACKING_URI=http://host.containers.internal:5000 \
    MIBICI_MODEL_NAME=mibici-equipo46-p1-demanda \
    MIBICI_MODEL_ALIAS=champion

WORKDIR /app

RUN apt-get update \
    && apt-get install --no-install-recommends -y libgomp1 \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt ./requirements.txt
RUN python -m pip install --no-cache-dir --disable-pip-version-check -r requirements.txt

RUN groupadd --system --gid 10001 app \
    && useradd --system --uid 10001 --gid app --create-home app

COPY --chown=10001:10001 api/ ./api/
COPY --chown=10001:10001 mibici/ ./mibici/

USER 10001:10001

EXPOSE 8000

CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

El modelo no se copia dentro de la imagen. Al arrancar, la API lo resuelve desde MLflow con el alias `champion`; por ello el contenedor necesita conectividad con el Registry y acceso a los artefactos. La estrategia y el montaje local se detallan en los puntos siguientes.

10. Punto 5 - Construcción y prueba local

La imagen se construyó con Docker Desktop desde la raíz del repositorio. El servicio requiere que MLflow esté activo en el puerto 5000 y que apunte a la misma base del Registry y al almacenamiento de artefactos que contienen el modelo registrado. En Windows se monta el directorio de modelos en la ruta Linux equivalente que espera la URI del artefacto.

10.1. Construir la imagen

```
docker build -t mibici-equipo46-api:semana8 -f Containerfile .
```

10.2. Levantar el contenedor

```
$modelsHost = "C:/IVAN/MNA/TC5061/semana-07/mlruns/2/models"
$modelsTarget = "/C:/IVAN/MNA/TC5061/semana-07/mlruns/2/models"

docker run --detach --rm --name mibici-equipo46-api `
  --publish 8000:8000 `
  --add-host host.containers.internal:host-gateway `
  --mount "type=bind,source=$modelsHost,target=$modelsTarget" `
  mibici-equipo46-api:semana8
```

La opción `--add-host` permite que Docker Desktop resuelva `host.containers.internal`. El bind mount expone los artefactos locales de MLflow al contenedor. El servicio queda publicado en `http://localhost:8000`.

10.3. Comprobaciones registradas

Comprobación	Resultado observado	Qué valida
GET /health	HTTP 200; `model_loaded: true`	El modelo champion se cargó desde MLflow.
POST /predict	HTTP 200; predicción **3.9233**	La petición ejecutó inferencia dentro del contenedor.
POST /predict con `target_hour = 24`	HTTP 422	La API rechaza la hora fuera de rango antes de invocar el modelo.

Petición de prueba válida y respuesta obtenida:

POST /predict - JSON de entrada

```
{
  "station_id": 1,
  "departures_this_hour": 5,
  "arrivals_this_hour": 4,
  "departures_mean_last_24h": 3.5,
  "departures_mean_last_168h": 4.2,
  "target_hour": 9,
  "target_dayofweek": 1,
  "target_is_weekend": 0
}
```

Respuesta JSON

```
{
  "station_id": 1,
  "predicted_departures_next_hour": 3.9233,
  "model_name": "mibici-equipo46-p1-demanda",
  "model_version": "1"
}
```

La respuesta coincide, redondeada a cuatro decimales, con la predicción 3.923333879487776 registrada al verificar el modelo real en la sección 8. La versión devuelta fue la 1.

11. Punto 6 - README y estrategia de despliegue

El **README.md** de la raíz del proyecto se actualizó para explicar el servicio completo y permitir que otro integrante lo construya, lo ejecute y lo consuma. La guía cubre los pasos 1 a 6 de la actividad y mantiene los detalles del modelo y de la API enlazados a las notas técnicas de Semana 8.

Contenido documentado en README.md	Instrucción o decisión
Construcción	Requisitos de Docker Desktop y comando `docker build` usando el Containerfile.
Conexión a MLflow	URI del tracking server, misma base del Registry y montaje de artefactos locales.
Ejecución	Comando `docker run`, publicación del puerto 8000, alias del host y montaje de modelos.
Consumo	Comprobaciones de `/health` y `/model-info`, ejemplo de `POST /predict` con respuesta JSON y validación HTTP 422.
Estrategia	Justificación de inferencia en tiempo real y límites de la configuración local.

11.1. Estrategia elegida: inferencia en tiempo real

Se eligió inferencia en tiempo real porque la API entrega una predicción inmediata por petición HTTP, adecuada para consultar la demanda de una estación en una hora objetivo. FastAPI carga una sola vez el modelo `champion` al iniciar y lo mantiene en memoria, evitando consultar el Registry en cada solicitud. El endpoint `/predict/batch` permite enviar hasta 1000 estaciones en una sola llamada; no implementa trabajos fuera de línea ni una cola de procesamiento.

Alcance operativo

Esta entrega es una ejecución académica local. El modelo y sus artefactos permanecen en MLflow fuera de la imagen. Para ejecutar el contenedor en otro equipo se debe habilitar un Registry con el alias y los artefactos, y ajustar la URI y el montaje a sus rutas; alternativamente se puede empaquetar un modelo exportado de versión controlada. El README identifica estos requisitos para evitar que una imagen construida sin acceso al modelo se confunda con un servicio listo para producir predicciones.

Los cambios de Containerfile y README están en la rama `feature/semana8-docker-container`.

12. Referencias técnicas

- MLflow. (s. f.). ML model registry. https://mlflow.org/docs/latest/ml/model-registry/
- MLflow. (s. f.). Deploy MLflow model as a local inference server. https://mlflow.org/docs/latest/ml/deployment/deploy-model-locally/
- FastAPI. (s. f.). Request body. https://fastapi.tiangolo.com/tutorial/body/
- Podman. (s. f.). Podman documentation. https://docs.podman.io/en/latest/
- pytest development team. (s. f.). pytest documentation. https://docs.pytest.org/

Anexo A. Recuperación y registro del modelo

Capturas de la ejecución local y de la interfaz de MLflow (7 de noviembre de 2026).

```
False
(.venv) PS C:\Users\axelr\Documents\MaestriaIA\TC5061-Proyecto-Equipo46> Test-Path .\mibici\serving\register_p1.py
True
(.venv) PS C:\Users\axelr\Documents\MaestriaIA\TC5061-Proyecto-Equipo46> $env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
>> $env:MLFLOW_REGISTRY_URI = "http://127.0.0.1:5000"
>>
>> python -m mibici.serving.register_p1
2026-11-07 21:42:44.393 | INFO     | mibici.config:<module>:69 - PROJ_ROOT: C:\Users\axelr\Documents\MaestriaIA\TC5061-Proyecto-Equipo46
Seleccionado: P1-HGB-02 | MAE validación=0.735193 | Run ID=f097ad4116364c8fb6182562cdf685f7
Origen: models:/m-a0c2d2829ffb4f5e9ec9b0fdd52e7a31
Versión de datos: 129a4a57efc7
Successfully registered model 'mibici-equipo46-p1-demanda'.
2026/11/07 21:42:48 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for model version to finish creation. Mode
l name: mibici-equipo46-p1-demanda, version 1
Created version '1' of model 'mibici-equipo46-p1-demanda'.
OK: mibici-equipo46-p1-demanda | versión=1 | alias=champion
Modelo P1 registrado y recuperable desde MLflow Model Registry.
(.venv) PS C:\Users\axelr\Documents\MaestriaIA\TC5061-Proyecto-Equipo46>
```

Figura A1. Registro del candidato P1-HGB-02 como versión 1 y alias champion.

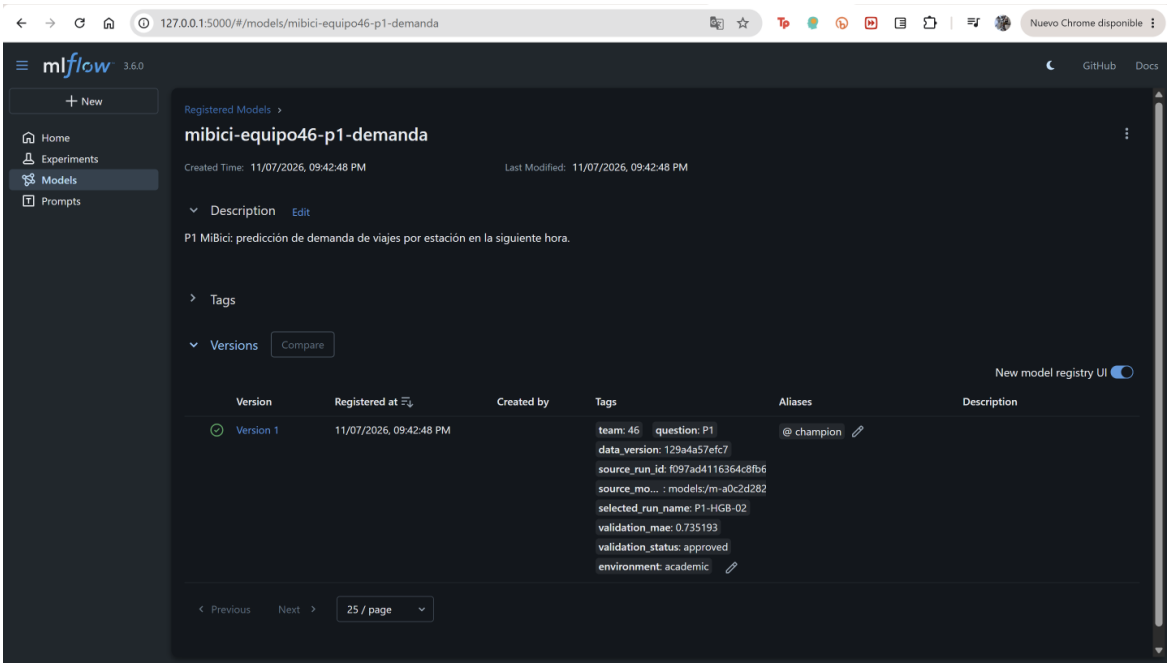


Figura A2. Model Registry: versión 1, alias champion y metadatos del modelo.

Anexo B. Predicción real y pruebas de integración

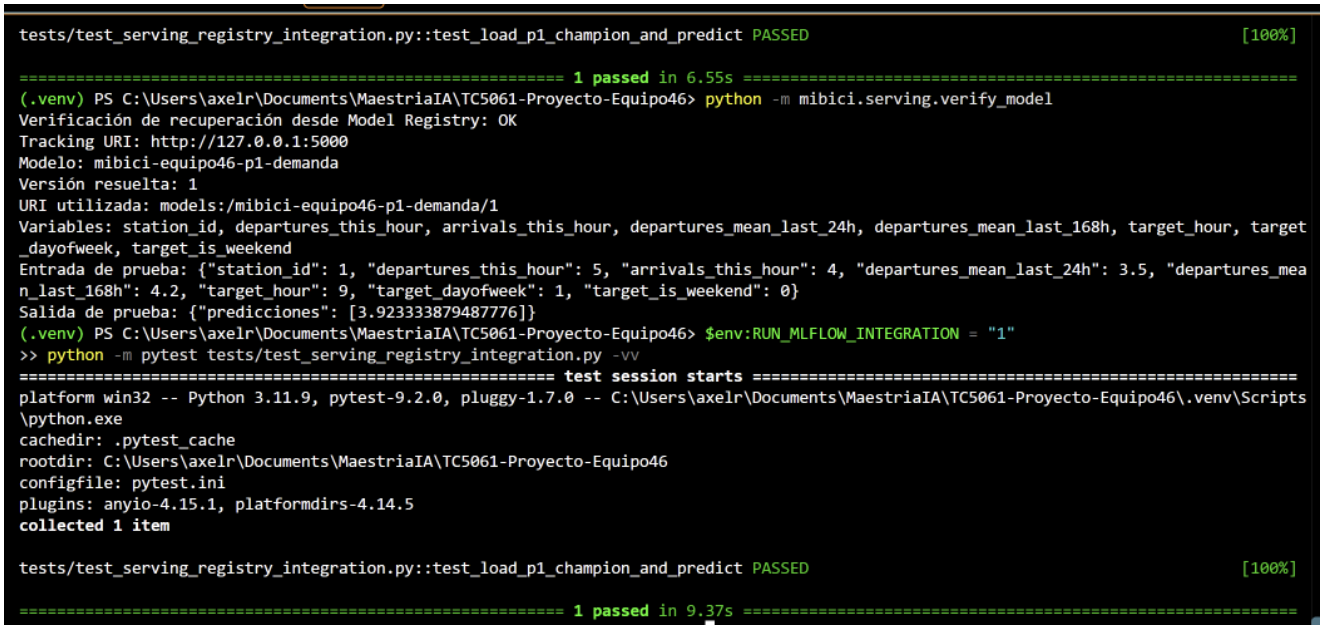


Figura B1. Carga desde Model Registry, predicción numérica y prueba de integración aprobada.

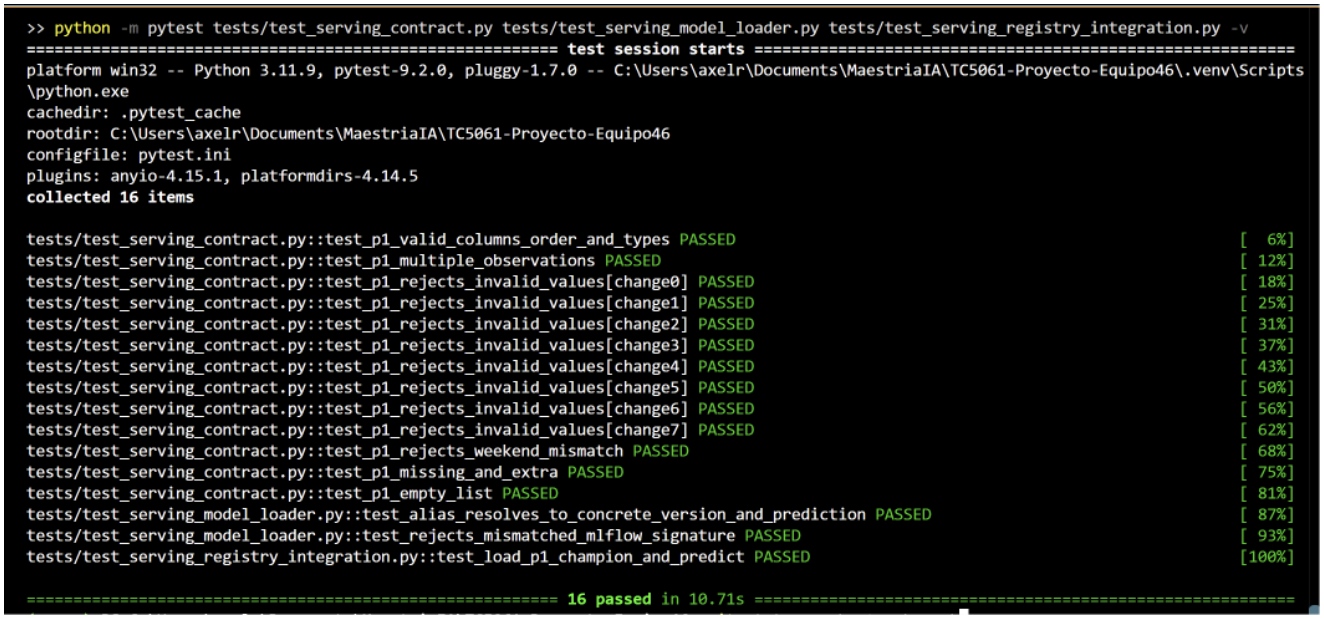


Figura B2. Validación de las 16 pruebas específicas del componente de serving.

Anexo C. Reproducibilidad y dependencias

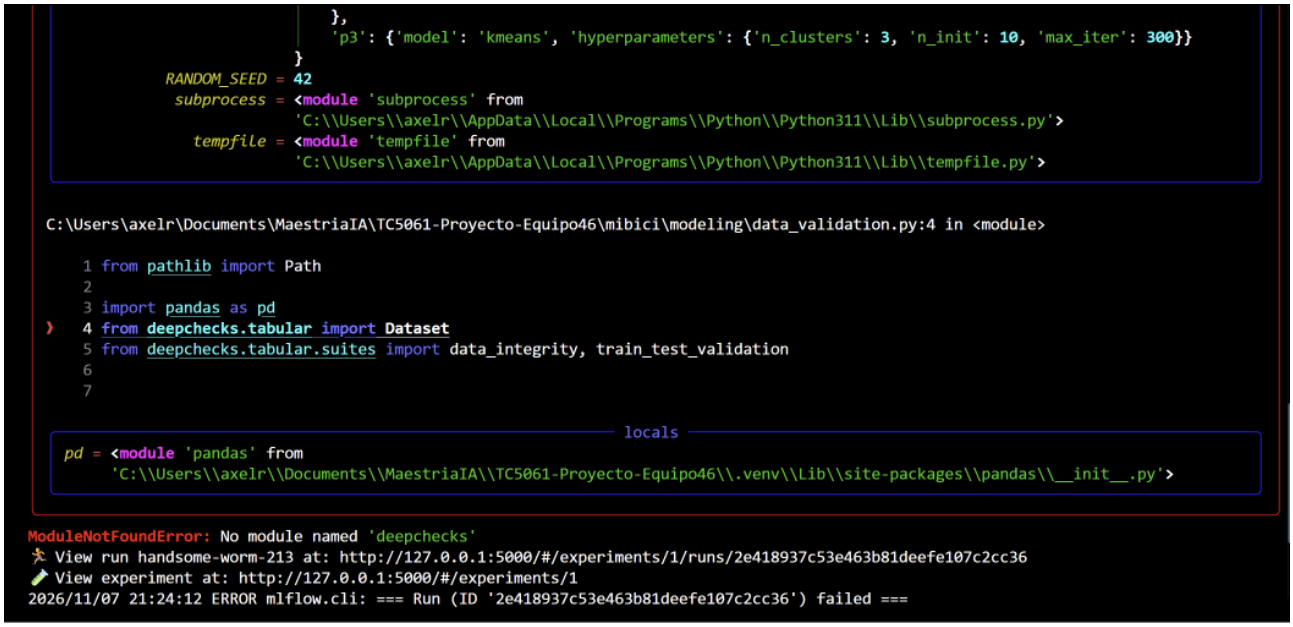


Figura C1. Error de dependencia detectado durante la reproducción del entrenamiento: falta Deepchecks.

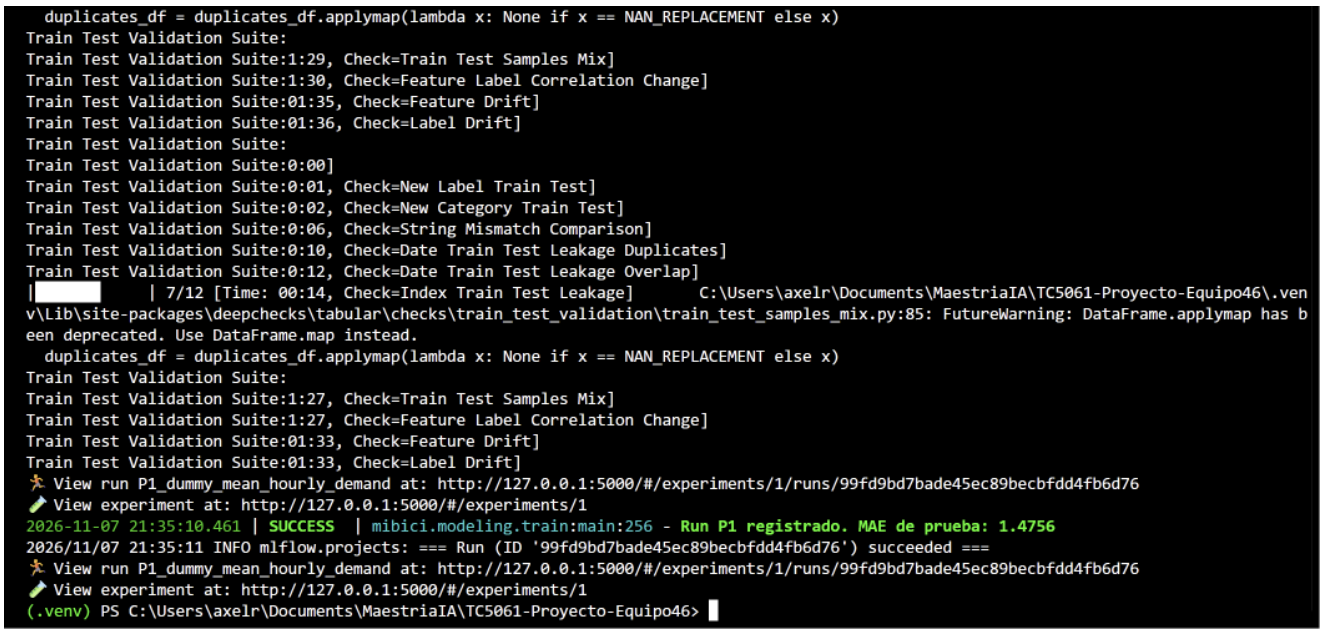


Figura C2. Entrenamiento P1 base completado y registrado en MLflow; MAE de prueba 1.4756.

Anexo D. Datos y entorno validado

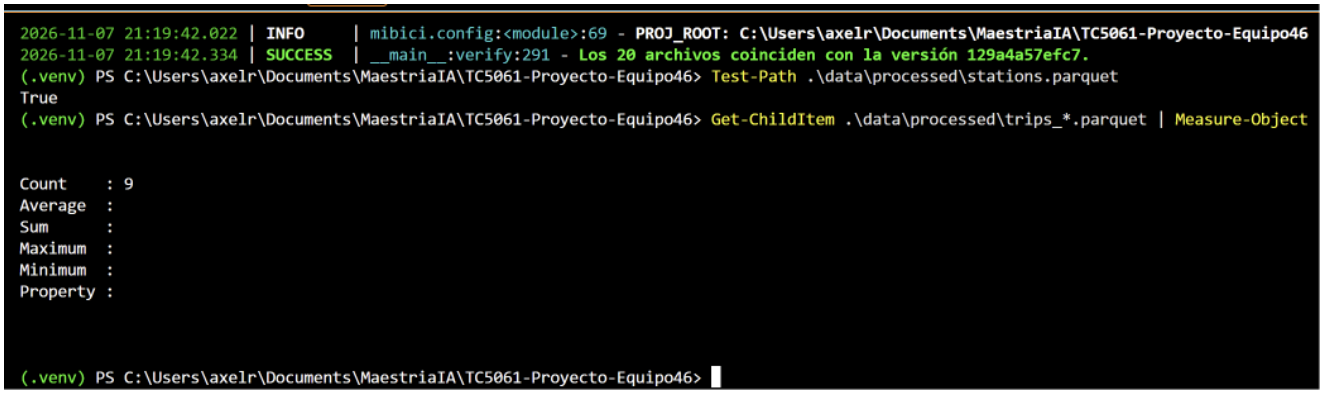


Figura D1. Integridad: 20 archivos coincidentes y nueve Parquet mensuales procesados.

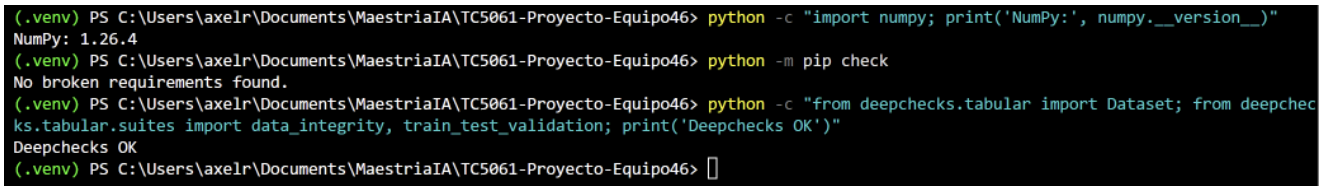


Figura D2. NumPy 1.26.4, Deepchecks OK y pip check sin conflictos.

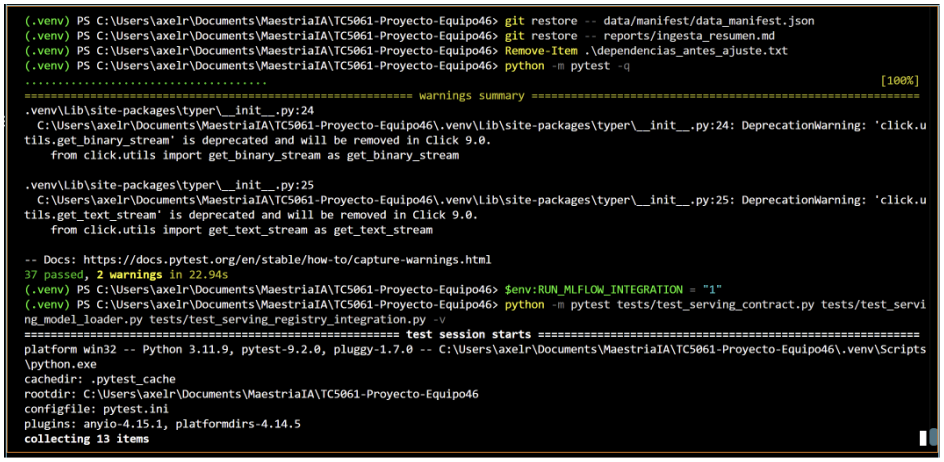


Figura D3. Ejecución general: 37 pruebas aprobadas y dos advertencias.

Anexo E. Evidencia de colaboración en GitHub

```
========================================================== 16 passed in 5.33s ==========================================================
(.venv) PS C:\Users\axelr\Documents\MaestriaIA\TC5061-Proyecto-Equipo46> git status --short --branch
## feature/semana8-mlflow-axel
 M mibici/config.py
 M requirements.txt
?? configs/empty.json
?? docs/Semana8_Axel_ModelRegistry.md
?? docs/evidencias/
?? mibici/serving/
?? tests/test_serving_contract.py
?? tests/test_serving_model_loader.py
?? tests/test_serving_registry_integration.py
(.venv) PS C:\Users\axelr\Documents\MaestriaIA\TC5061-Proyecto-Equipo46> git ls-files --others --exclude-standard
configs/empty.json
docs/Semana8_Axel_ModelRegistry.md
docs/evidencias/semana8/data_manifest_129a4a57efc7.json
mibici/serving/__init__.py
mibici/serving/contract.py
mibici/serving/model_loader.py
mibici/serving/register_p1.py
mibici/serving/verify_model.py
tests/test_serving_contract.py
tests/test_serving_model_loader.py
tests/test_serving_registry_integration.py
(.venv) PS C:\Users\axelr\Documents\MaestriaIA\TC5061-Proyecto-Equipo46> git diff --stat
 mibici/config.py | 2 +-
```

Figura E1. Identificación de archivos y cambios de la rama feature/semana8-mlflow-axel.



Figura E2. Pull Request #12 abierto desde la rama de Axel hacia main, con 13 archivos.

Anexo F. API REST con FastAPI

```
(.venv311) PS C:\Users\monic\Desktop\TC5061-Equipo46-seguridad> python -m pytest tests/test_api.py -v
================================== test session starts ==================================
platform win32 -- Python 3.11.9, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\monic\Desktop\TC5061-Equipo46-seguridad\.venv311\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\monic\Desktop\TC5061-Equipo46-seguridad
configfile: pytest.ini
plugins: anyio-4.16.0, platformdirs-4.14.5
collected 15 items

tests/test_api.py::test_schema_matches_model_contract PASSED                      [  6%]
tests/test_api.py::test_health_ok PASSED                                          [ 13%]
tests/test_api.py::test_model_info PASSED                                         [ 20%]
tests/test_api.py::test_predict_ok PASSED                                         [ 26%]
tests/test_api.py::test_predict_rejects_invalid_input[change0] PASSED             [ 33%]
tests/test_api.py::test_predict_rejects_invalid_input[change1] PASSED             [ 40%]
tests/test_api.py::test_predict_rejects_invalid_input[change2] PASSED             [ 46%]
tests/test_api.py::test_predict_rejects_invalid_input[change3] PASSED             [ 53%]
tests/test_api.py::test_predict_rejects_invalid_input[change4] PASSED             [ 60%]
tests/test_api.py::test_predict_rejects_invalid_input[change5] PASSED             [ 66%]
tests/test_api.py::test_predict_rejects_invalid_input[change6] PASSED             [ 73%]
tests/test_api.py::test_predict_rejects_missing_field PASSED                      [ 80%]
tests/test_api.py::test_batch PASSED                                              [ 86%]
tests/test_api.py::test_batch_rejects_empty_list PASSED                           [ 93%]
tests/test_api.py::test_unavailable_model_returns_503 PASSED                     [100%]

================================== warnings summary ==================================
.venv311\Lib\site-packages\starlette\testclient.py:51
  C:\Users\monic\Desktop\TC5061-Equipo46-seguridad\.venv311\Lib\site-packages\starlette\testclient.py:51: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use an
yio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================== 15 passed, 1 warning in 9.74s ==============================
(.venv311) PS C:\Users\monic\Desktop\TC5061-Equipo46-seguridad>
feature/semana8-api-fastapi*  ⊗ 0 ⚠ 0  Pending package refresh
```

Figura F1. Pruebas de la API: 15 aprobadas con Python 3.11.9.

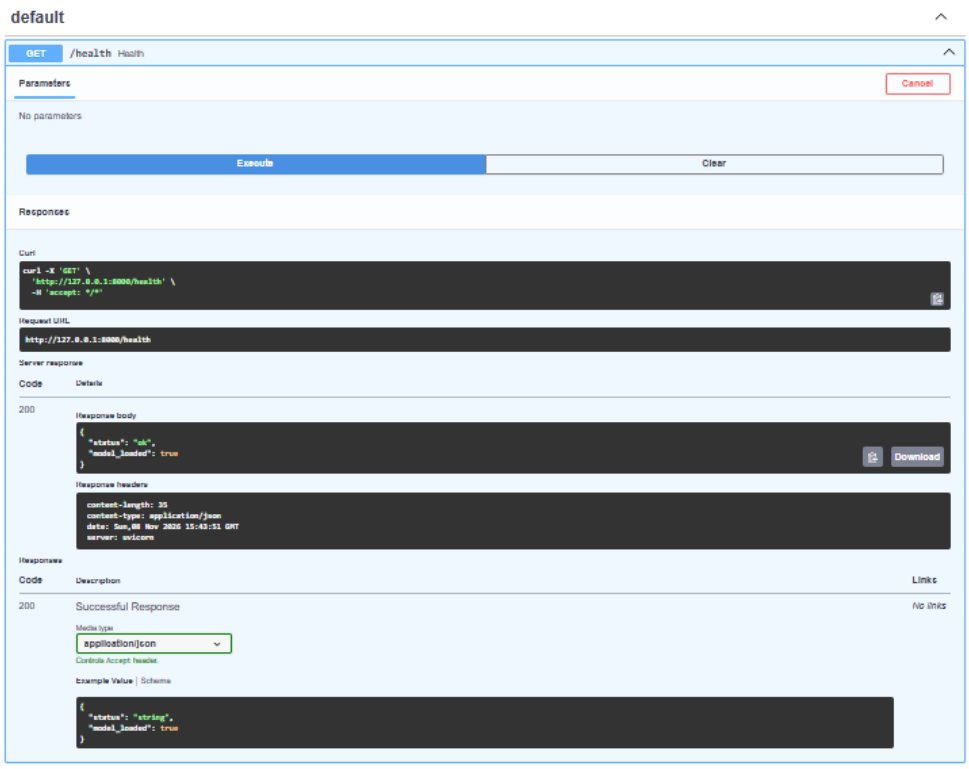


Figura F2. GET /health: modelo cargado desde MLflow Model Registry.

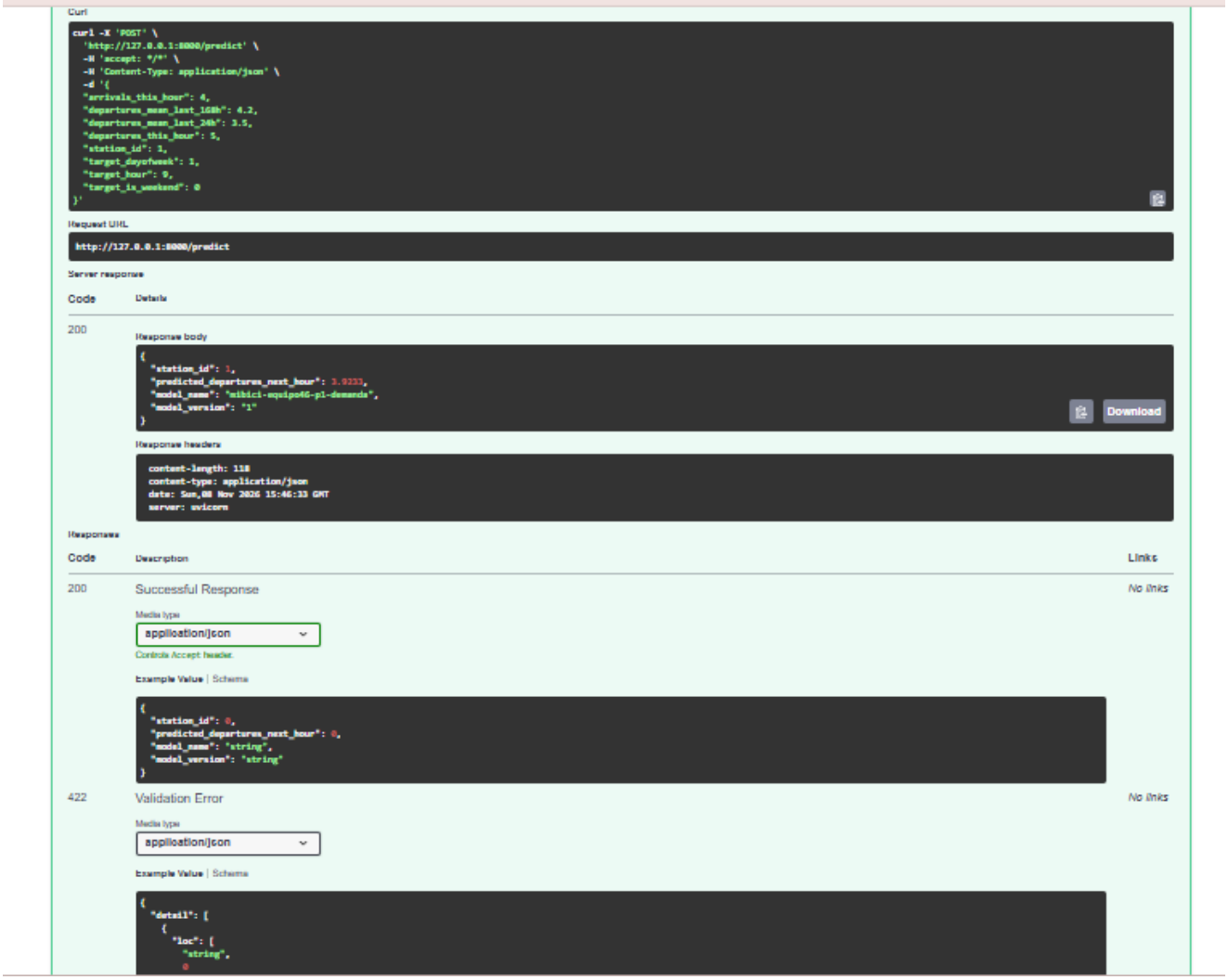


Figura F3. POST /predict con el modelo real: predicción 3.9233, versión 1.

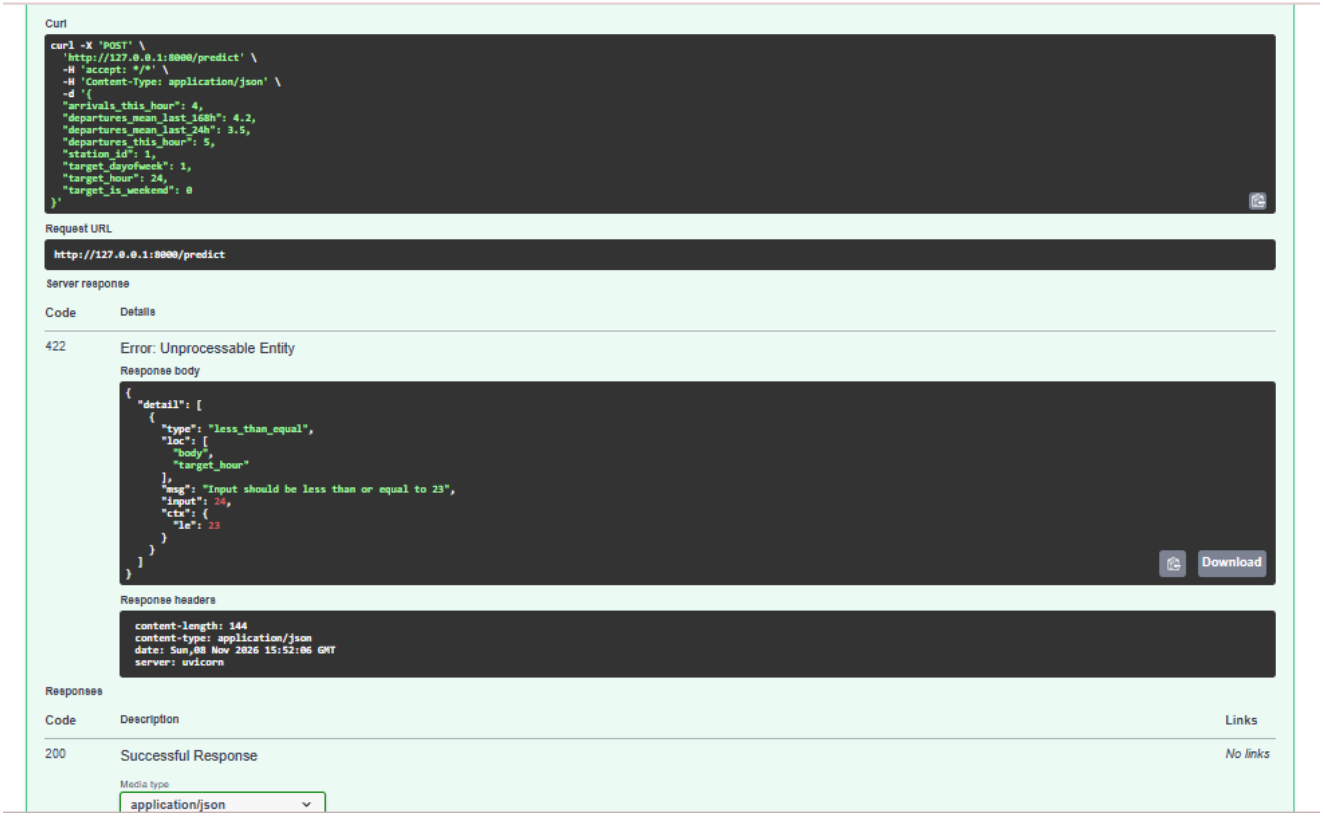


Figura F4. POST /predict rechaza target_hour = 24 con HTTP 422.

Anexo G. Evidencias de los puntos 4 y 5

Figura G1. Containerfile en la raíz del repositorio: dependencias, usuario no privilegiado, puerto 8000 y comando de Uvicorn (punto 4).

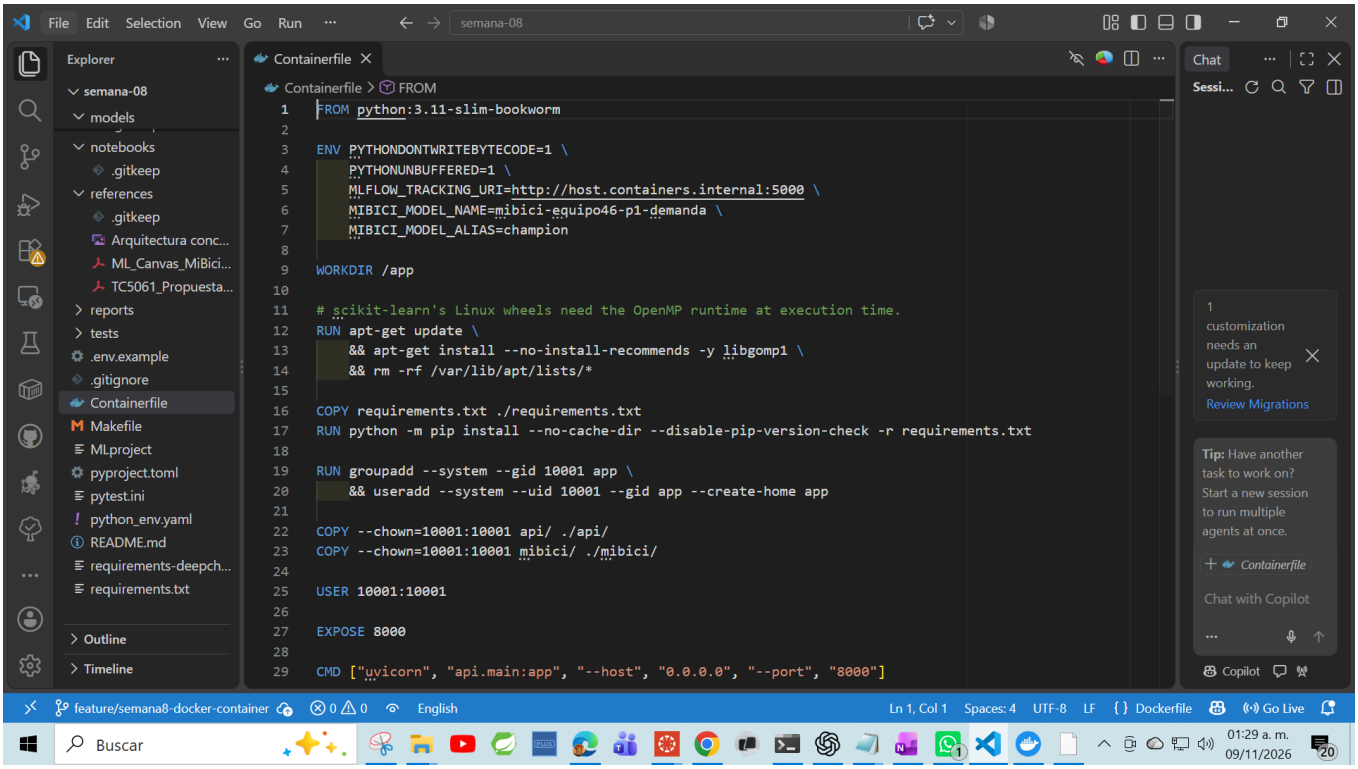
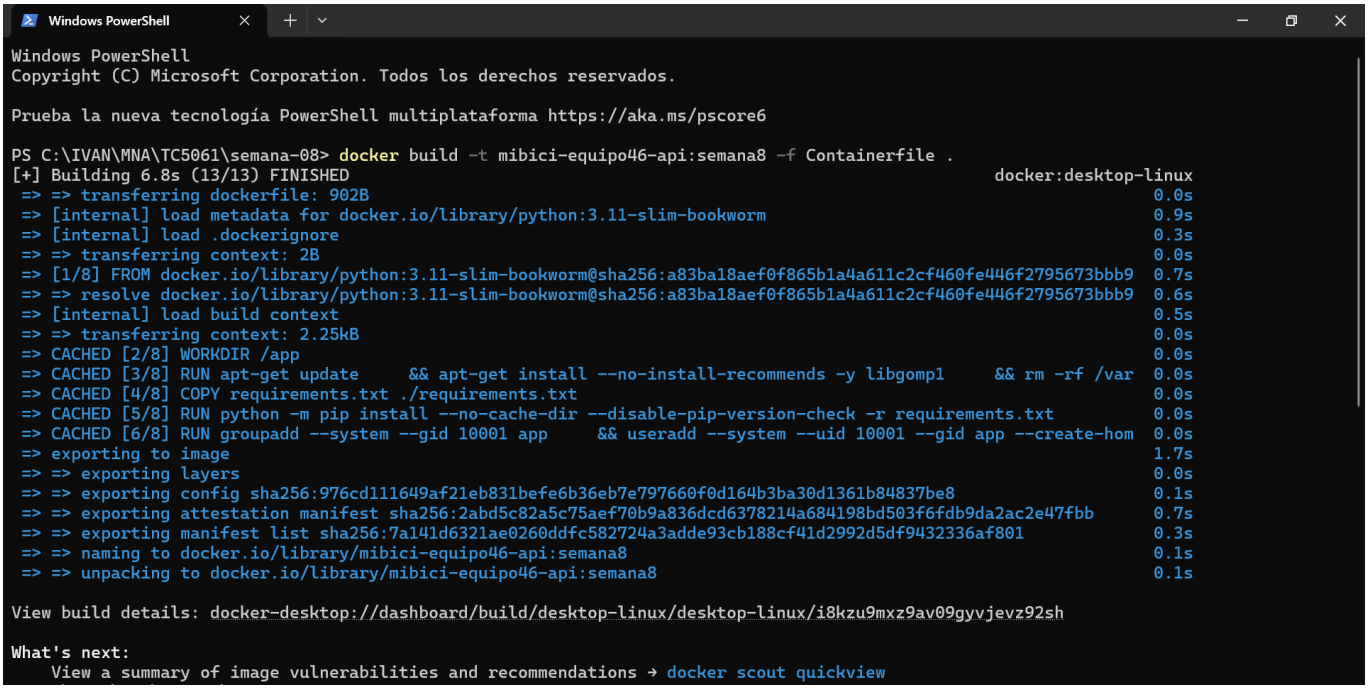


Figura G2. Construcción finalizada (13/13) y etiqueta de imagen `mibici-equipo46-api:semana8` *(punto 5).*



Anexo G. Evidencias de contenedor y servicio

Figura G3. Contenedor en ejecución, imagen de la API y mapeo del puerto 8000:8000 (punto 5).

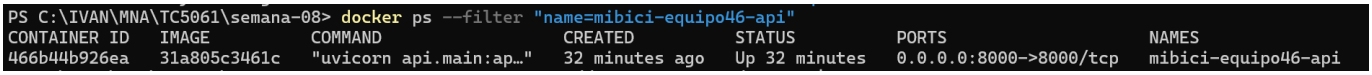
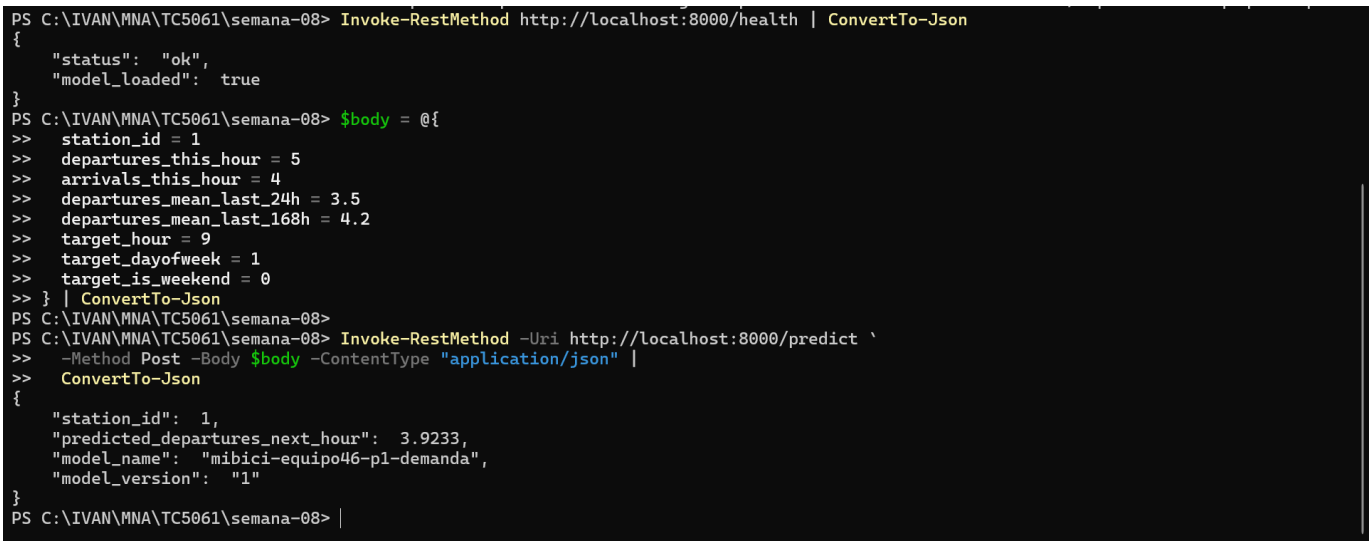


Figura G4. Respuesta de `/health` con el modelo cargado y predicción de `/predict` igual a 3.9233 (punto 5).



Punto 6 - Documentación del servicio. El README de la raíz del repositorio contiene las instrucciones para construir y ejecutar la imagen, un ejemplo de petición y respuesta, y la justificación de inferencia en tiempo real. El cuerpo de este informe resume esa documentación en la sección 11. El README actualizado está en la rama `feature/semana8-docker-container`.